

Timothy Gunn
Malik Mashigo

Reinforcement Learning for Adaptive Optimal Execution

The primary goal of our project was to minimize slippage, or the difference between the price at which a trading decision is made and the final average execution price. When executing large orders, slippage can significantly impact profitability. Furthermore, current order execution algorithms used to address this problem, such as Time-Weighted Average Price (TWAP) and Volume-Weighted Average Price, are static and predictable. As a result, other market participants could detect their behavior and exploit it. Additionally, these algorithms do not adapt to changing market conditions in real time.

To address these issues, we developed a reinforcement learning agent capable of reacting to the real time state of the order book and deciding how to optimally execute large orders. Unlike TWAP and VWAP, the bot learned to balance slippage (the cost of trading too aggressively) and the opportunity cost (the cost of not trading at all).

Data Collection

Our biggest challenge while completing the project was data collection. Since the bots on NDFEX were not configured to behave like realistic market participants for most of the semester, we could not simply use that data for our agent. Instead, we considered other alternatives such as purchasing access to live data from an actual market and downloading large scale historical datasets, which are often terabytes in size. Given the practical constraints of both approaches, we decided to focus on creating the reinforcement learning environment and validating that it worked before collecting data at scale. As a result, we created a data collector that subscribes to NDFEX and gathered a small amount of data to ensure that the training pipeline functioned correctly. Then once the exchange started functioning normally we collected data for much longer in hopes of exposing the bot to a variety of market conditions.

Methodology

We implemented the simulation environment as a custom OpenAI Gymnasium environment that replays historical order book data. At each timestep, the agent observes a seven dimensional state vector consisting of remaining inventory percentage, remaining time percentage, bid-ask spread, L1 and L3 order book imbalance, and normalized bid and ask quantities. The agent then selects from five discrete aggression levels, ranging from doing nothing to immediately submitting a market order. The reward function penalizes implementation shortfall at each step and applies an additional penalty for any unexecuted inventory at the last timestep so that the agent would learn to balance execution cost and completion risk.

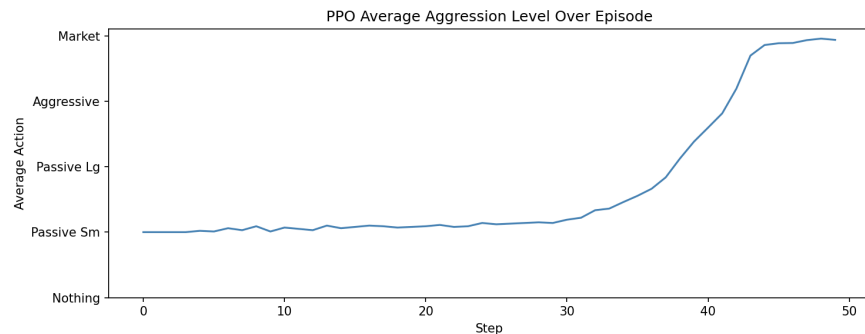
The agent was trained using Proximal Policy Optimization (PPO), which is a reinforcement learning algorithm that uses actor-critic architecture. The actor, otherwise known as the policy network, maps the state vector (the input) to a probability distribution over the five possible actions. The critic, otherwise known as the value network, maps the same state vector to a scalar estimate of expected future reward, which is used to compute the advantage and guide policy updates.

Results

The PPO agent significantly outperformed the other baselines:

Strategy	Mean IS	Std IS	Percent Filled
TWAP	.0205	.0179	99.97%
VWAP	.0150	.0144	99.27%
PPO	.0087	.0114	99.87%

The agent reduced mean implementation shortfall by 58% relative to TWAP and 43% relative to VWAP, while also achieving smaller variance and a near perfect fill rate.



One of the most notable behaviors of the bot was its ability to dynamically adapt execution aggression based on remaining time. Early in the execution window, the bot traded more conservatively to minimize market impact. However, as the execution deadline approached the agent learned to become increasingly aggressive to ensure completion of the order. This adaptive behavior emerged through reinforcement learning and represents a major advantage over static execution algorithms.

Future Work

Although the project produced promising results, there are several improvements that could make the system more realistic and robust. Most importantly, the current environment is a passive historical replay where the market does not react to the agent's actions. In reality, other market participants would respond to the bot's orders. As a result, one major enhancement would be the creation of an adversarial simulation in which other market participants actively react to the bot's behavior. Such a simulation would prevent the agent from exploiting fixed fill probabilities and force it to learn more varied and realistic execution strategies.

In addition to an adversarial simulation, we are currently working on retraining the model using data collected from an actual exchange rather than simulated NDFEX data. Since the agent will be exposed to a broader range of market conditions, we expect it to generalize better and become more robust overall.

Conclusion

Overall, this project provided us with a unique opportunity to apply reinforcement learning to a real financial problem. We learned how to collect and process order book data, design an execution environment, and create a reward function capable of producing meaningful agent behavior. We also gained practical experience with proximal policy optimization and actor-critic architectures.

Most importantly, this project demonstrated that reinforcement learning can provide substantial advantages over traditional static execution algorithms by teaching the agent how to adapt to changing market conditions. While there are still many improvements to be made, the results suggest that reinforcement learning has significant potential for optimal execution in the market.